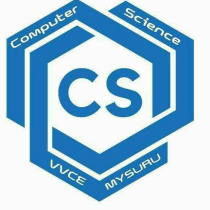


Vidyavardhaka College of Engineering, Mysuru

Autonomous Institute, Affiliated to VTU

Accredited by NBA | NAAC with 'A' Grade



Blockchain Technology- BTCS603

Module 3- Asymmetric Cryptography

6th **SEM**

Lakshmi B S
Assistant
Professor
Dept. of CSE

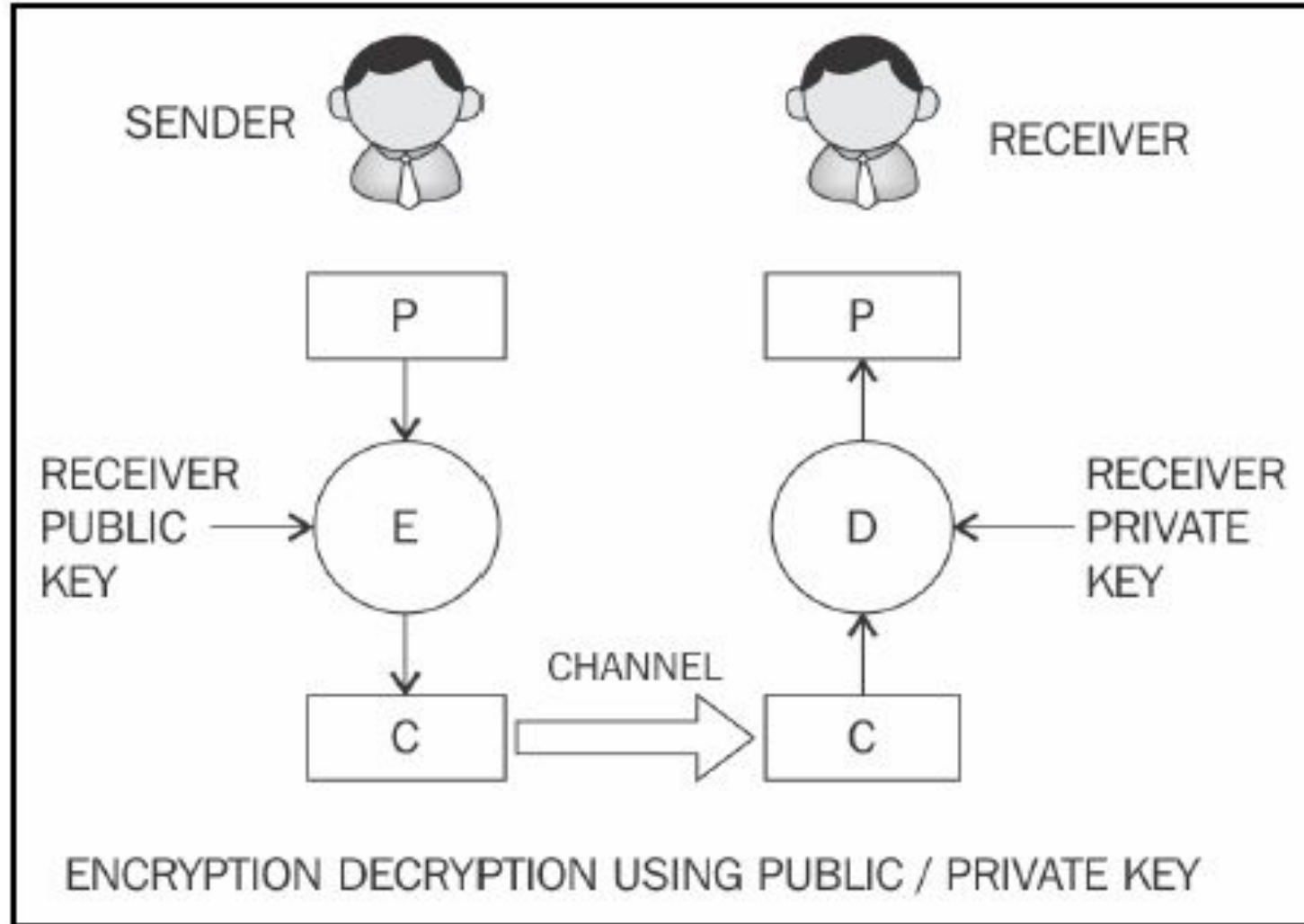
Our Vision: "VVCE shall be a leading Institution in engineering and management education enabling individuals for significant contribution to the society"

Topics to be covered...

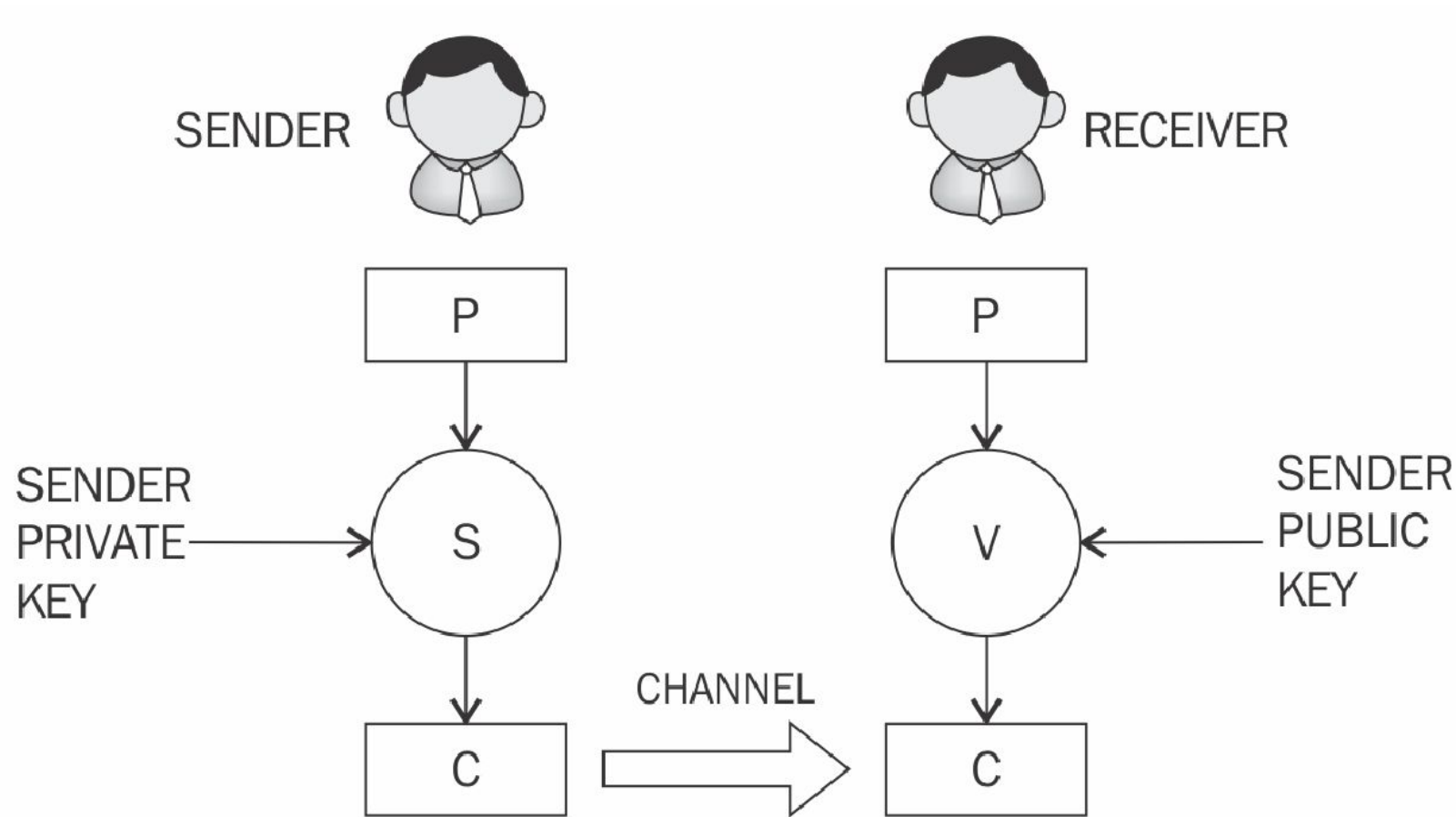
- **Cryptographic primitives:**

- Asymmetric cryptography,
- Hash functions,
- Secure Hash Algorithms (SHAs),
- Merkle trees,
- Patricia trees,
- Distributed hash tables (DHTs),
- Digital signatures.

Asymmetric cryptography:



Model is only used for message authentication and validation purposes:



SIGNING & VERIFICATION USING PUBLIC / PRIVATE KEY

- Security mechanisms offered by public key cryptosystem include
 - Key establishment---- setting of Keys over a insecure channel
 - Digital signatures ---- Non-repudiation service
 - Identification----- Identify the entity involved in transaction
 - Encryption, and decryption----- Confidentiality

1. Key Establishment

- **Purpose:** Securely exchanging encryption keys over an insecure channel.
- **Mechanism:** Public key cryptography enables secure key exchange
- **Example:**
 - **Diffie-Hellman Key Exchange:** Two parties securely establish a shared secret key over an insecure network.
 - **RSA-based Key Exchange:** A sender encrypts a secret key with the recipient's public key, which can only be decrypted with the recipient's private key.

2. Digital Signatures

- **Purpose:** Provides **authentication, integrity, and non-repudiation.**
- **Mechanism:**
 - The sender signs a message using their **private key**.
 - The receiver verifies the signature using the sender's **public key**.
- **Example:**
 - RSA, DSA, and ECDSA digital signatures ensure that a document/message comes from a legitimate sender and has not been tampered with.

3. Identification

- **Purpose:** Verifies the identity of an entity (e.g., user, device, or server) in a transaction.
- **Mechanism:**
 - A challenge-response mechanism using a public-private key pair.
 - The entity proves its identity by decrypting a challenge using its **private key**, and the verifier confirms it using the entity's **public key**.
- **Example:**
 - **SSL/TLS Handshake:** Websites prove their authenticity using **X.509 certificates** signed by Certificate Authorities (CAs).
 - **Smart Cards & Digital IDs:** Used for secure authentication.

4. Encryption and Decryption

- **Purpose:** Ensures **confidentiality** of data by converting plaintext into ciphertext.
- **Mechanism:**
 - The sender encrypts the data with the recipient's **public key**.
 - Only the recipient can decrypt it using their **private key**.
- **Example:**
 - **RSA Encryption:** Often used in secure email (PGP), SSL/TLS, and secure messaging.
 - **Elliptic Curve Cryptography (ECC):** A lightweight alternative to RSA for secure encryption.

Public and private keys

- A private key, as the names suggests, is basically a randomly generated number that is kept secret and held privately by the users.
- Private key needs to be protected and no unauthorized access should be granted to that key.
- A public key is available publicly and published by the private key owner. Anyone who would then like to send the publisher of the public key an encrypted message can do so by encrypting the message using the published public key and sending it to the holder of the private key.

- Public key cryptography algorithms are based on various underlying mathematical problems.
- There are three main families of asymmetric algorithms.

□ **Integer factorization.**

□ **Discrete logarithm**

□ **Elliptic curves**

Integer factorization.

- These schemes are based on the fact that large integers are very hard to factor. RSA is the prime example of this type of algorithm.

Discrete logarithm

- This is based on a problem in modular arithmetic that it is easy to calculate the result of modulo function, but it is computationally infeasible to find the exponent of the generator.
- In other words, it is extremely difficult to find the input from the result. This is a one-way function.
- For example, consider the following equation:

$$3^2 \bmod 10 = 9$$

- Now given 9 finding 2, the exponent of the generator 3 is very hard. This hard problem is
- commonly used in **Diffie-Hellman** key exchange and digital signature algorithms.

Elliptic curves

- Elliptic Curve Cryptography (ECC) is an asymmetric encryption technique based on the mathematical structure of elliptic curves. ECC provides the same level of security as RSA but with much smaller key sizes, making it efficient for cryptographic applications.

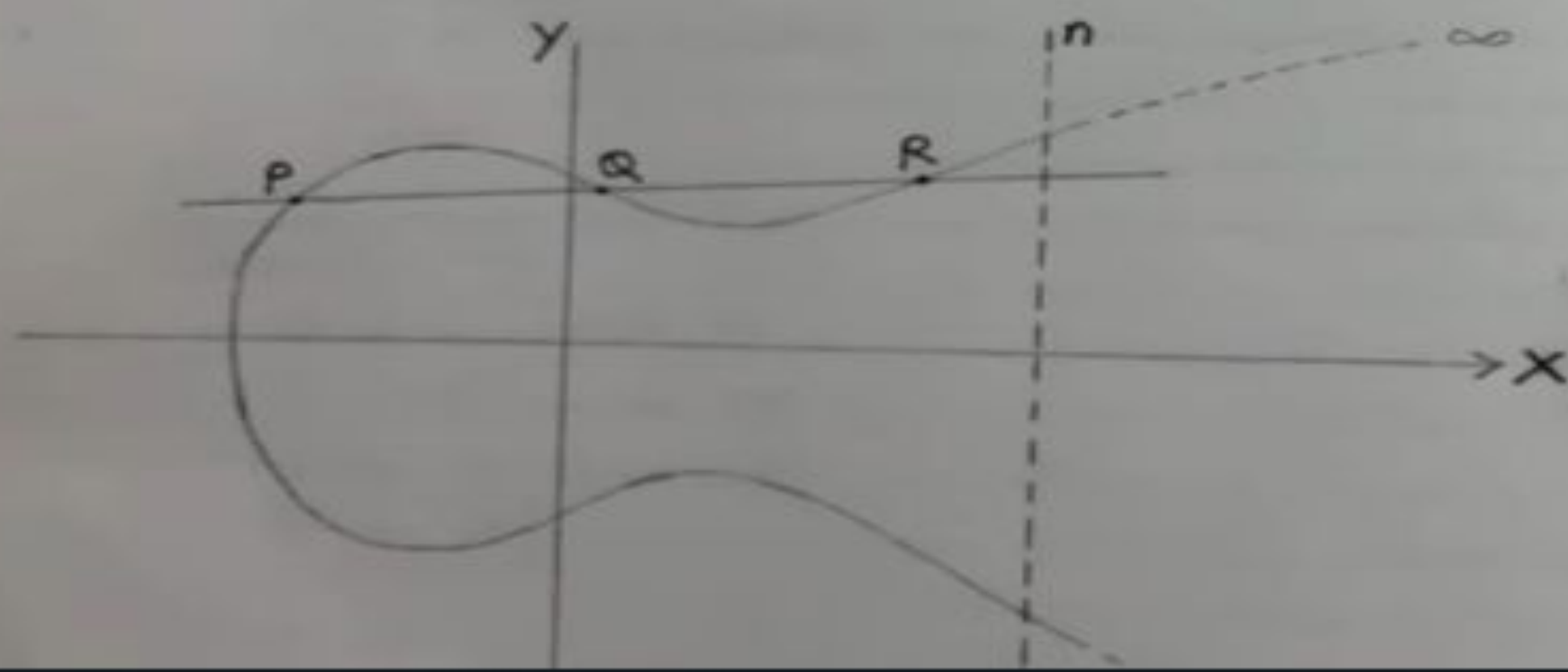
$$y^2 = x^3 + ax + b$$

- Elliptic curves can be defined over reals, rational numbers, complex numbers, or finite fields.
- For cryptographic purposes, elliptic curve over prime finite fields is used instead of real numbers. Additionally, the prime should be greater than 3.
- Different curves can be generated by varying the value of a , b .
- Mostly prominently used cryptosystems based on elliptic curves are **Elliptic Curve Digital Signatures Algorithm (ECDSA)** and **Elliptic Curve Diffie-Hellman (ECDH)** key exchange.

Elliptic Curve Cryptography (ECC)

- It is an asymmetric / public key cryptosystem
- It provides equal security with smaller key size as compared to non ECC algorithms
- It makes use of elliptic curves
- Elliptic curves are defined by some mathematical functions

$$E: y^2 = x^3 + ax + b$$



ECC Key Size (bits)	Equivalent RSA Key Size (bits)
160-bit ECC	1024-bit RSA
224-bit ECC	2048-bit RSA
256-bit ECC	3072-bit RSA
384-bit ECC	7680-bit RSA
521-bit ECC	15360-bit RSA

Global Public Elements -

1) $E_q(a, b)$ - Elliptic curve with parameters $a, b \in \mathbb{F}_q$ (prime number or an integer of the form 2^m)

2) G - Point on the elliptic curve

User A key generation

Select private key n_A $n_A < n$

Calculate public key P_A $P_A = n_A \times G$

User B key generation

Select private key n_B $n_B < n$

Calculate public key P_B $P_B = n_B \times G$

Calculation of secret key by user A

$$K = n_A \times P_B$$

Calculation of secret key by user B

$$K = n_B \times P_A$$

ECC Encryption

- Let the message be M
- First encode this message M into a point on elliptic curve

Let this point be P_m

- For encryption, choose a random positive integer k

The cipher point will be

$$C_m = \{ kG, P_m + kP_B \}$$

This point will be sent to the receiver

Decryption

- For decryption, multiply x -coordinate with receiver's secret key

$$kG \times n_B$$

Then subtract $(kG \times n_B)$ from y -coordinate of cipher point

$$P_m + kP_B - (kG \times n_B)$$

$$\text{we know that } P_B = n_B \times G$$

$$\therefore P_m + kP_B - kP_B$$

$$= P_m$$

So receiver gets the same point

Public and Private Keys

- A private key, as the names suggests, is basically a randomly generated number that is kept secret and held privately by the users. Private key needs to be protected and no unauthorized access should be granted to that key;
- A public key is available publicly and published by the private key owner. Anyone who would then like to send the publisher of the public key an encrypted message can do so by encrypting the message using the published public key and sending it to the holder of the private key

RSA

- RSA was invented in 1977 by *Ron Rivest*, *Adi Shamir*, and *Leonard Adelman*, hence the name RSA.
- This is based on the integer factorization problem, where the multiplication of two large prime numbers is easy but difficult to factor it back to the two original numbers.
- An RSA key pair is generated by performing the steps described here.

Modulus generation:

- Select p and q very large primes
- Multiply p and q , $n=p * q$ to generate modulus n

Generate co-prime:

- Assume a number called e .
- It should satisfy certain conditions, that is, it should be greater than 1 and less than $(p-1)(q-1)$

Generate public key:

- Modulus generated in step 1 and e generated in step 2 is pair that, together, is a
- public key. This part is the public part that can be shared with anyone; however,
- p and q need to be kept secret.

Generate private key:

- Private key called d here and is calculated from p , q and e . Private key is basically
- the inverse of e modulo $(p-1)(q-1)$. In the equation form, it is this:
- $ed = 1 \bmod (p-1)(q-1)$ also written as $de \bmod (p-1)(q-1) = 1$

Problems

- Given a Prime numbers 3 and 11, generate the private key and public key and encrypt and decrypt the message 2 using RSA Algorithm.

- Given a Prime numbers 3 and 11, generate the private key and public key and encrypt and decrypt the message hello using RSA Algorithm.

- Given Prime numbers are $p=7$ and $q=11$ and public key is $e = 7$, compute the private key and encrypt message called hello

- **OpenSSL Commands to Generate private key , public key and perform encryption and decryption.**

- Generate a Private Key (2048-bit RSA)

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt  
rsa_keygen_bits:2048
```

- Extract the Public Key from the Private Key

```
openssl rsa -in private_key.pem -pubout -out public_key.pem
```

Encryption and decryption

- Encrypt a Message Using the Public Key
- **Create a Plaintext File**

```
echo "Hello, this is a secret message!" > message.txt
```

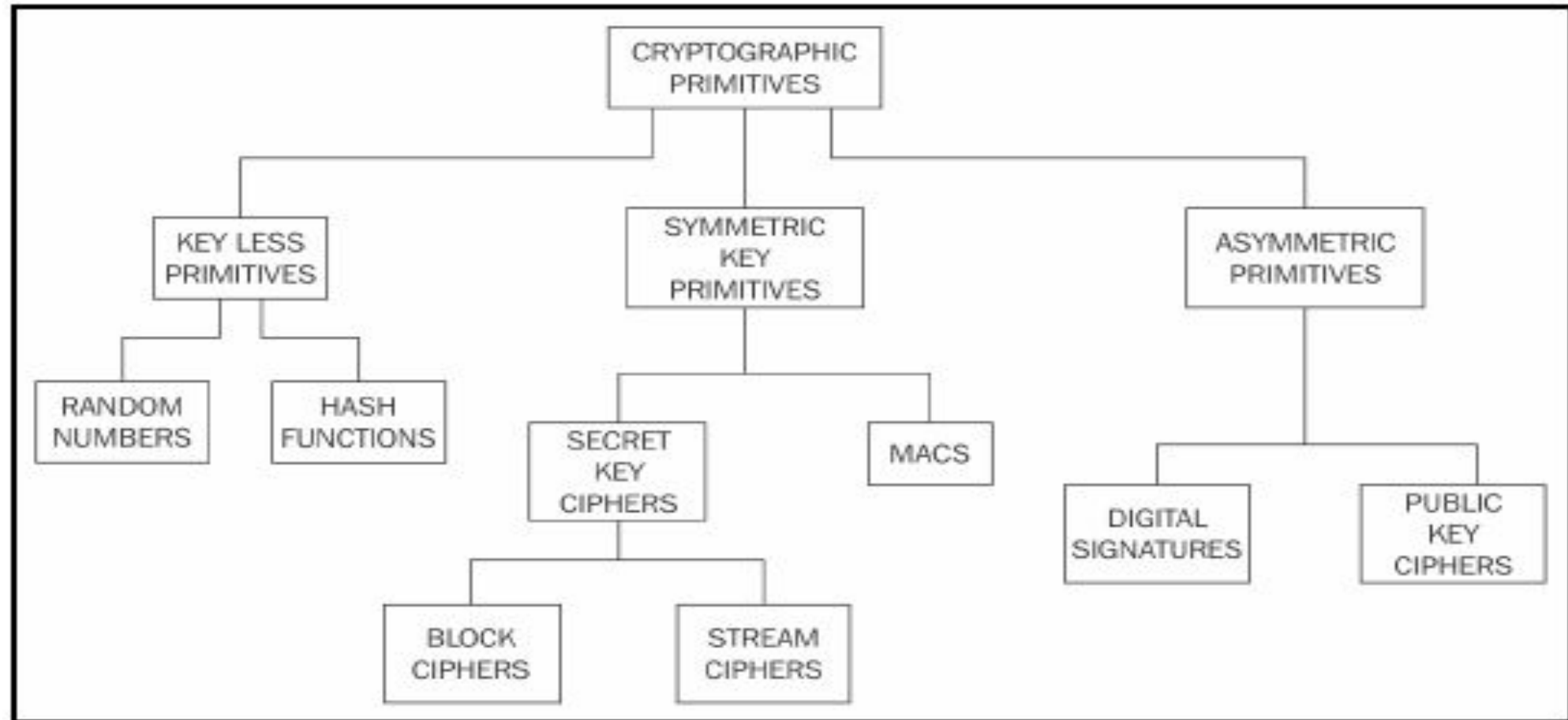
- **Encrypt the Message**

```
openssl rsautl -encrypt -inkey public_key.pem -pubin -in message.txt -out  
encrypted_message.bin
```

- **Decrypt the Message Using the Private Key**

```
openssl rsautl -decrypt -inkey private_key.pem -in encrypted_message.bin -out  
decrypted_message.txt
```

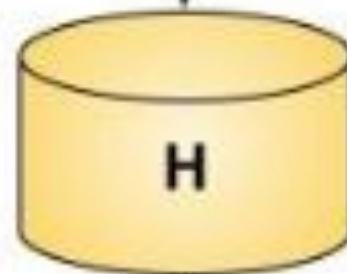
Cryptographic primitives



Hash functions

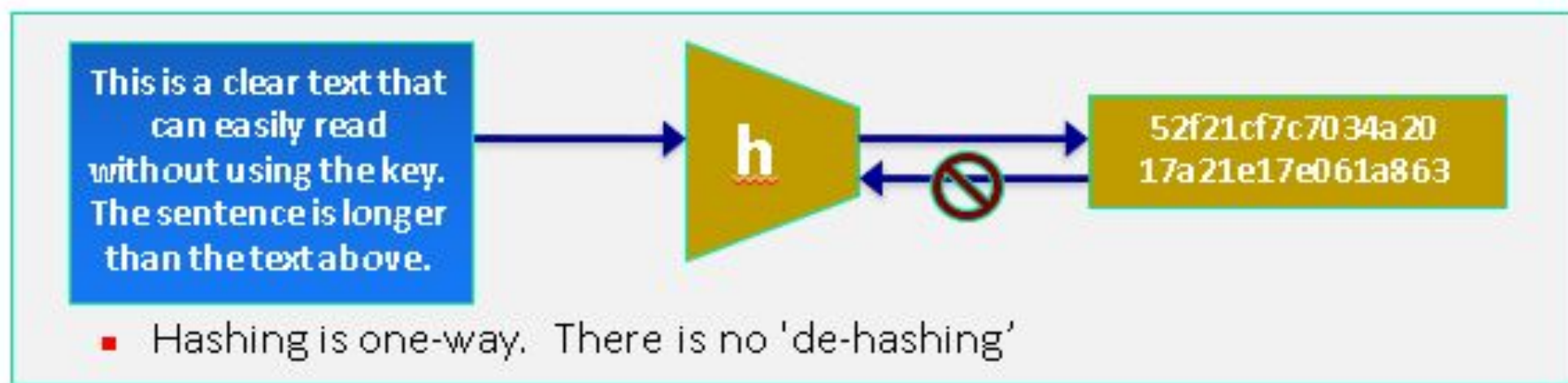
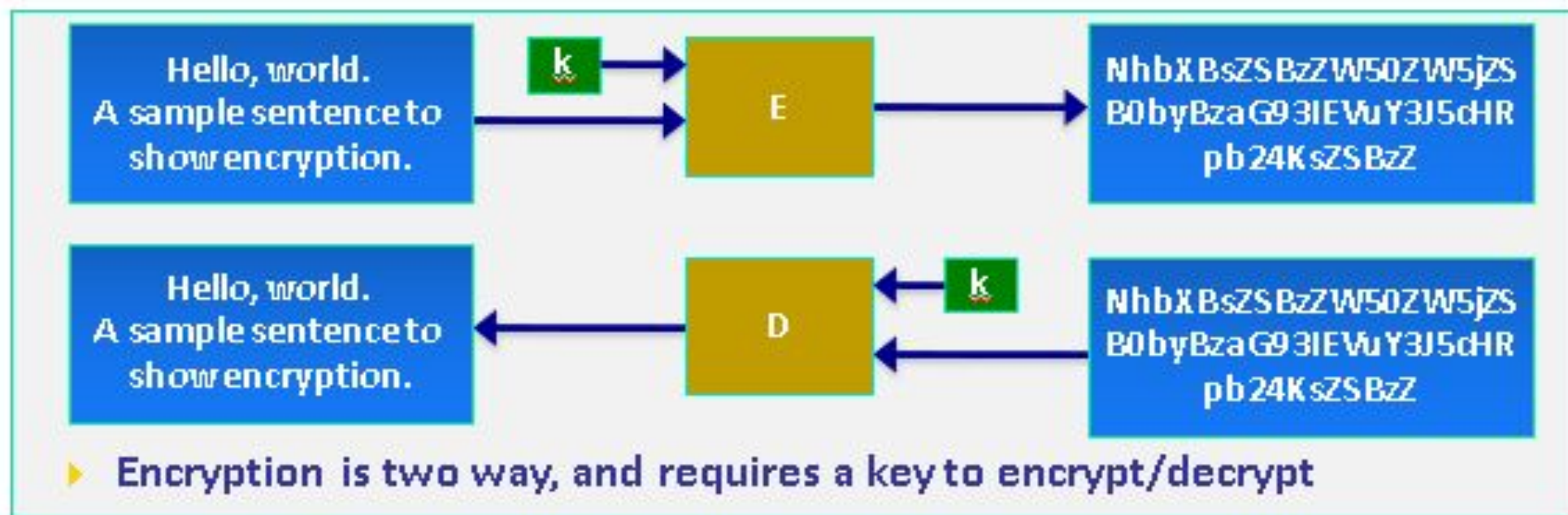
- Hash functions are used to create fixed length digests of arbitrarily long input strings.
- Hash functions are keyless and provide the data integrity service.
- Various families of hash functions are available, such as MD, SHA-1, SHA-2, SHA-3, RIPEMD, and Whirlpool.
- Hash functions are commonly used in digital signatures and message authentication codes, such as HMACs.
- They have three security properties, namely pre-image resistance, second pre-image resistance, and collision resistance.

Message M (arbitrary length)



Hash Value
(Fixed Length)

Hashing V.S. Encryption



Compression of arbitrary messages into fixed length digest

- This property is concerned with the fact that a hash function must be able to take a long input text of any length and output a fixed length compressed message.
- Hash functions produce a compressed output in various bit sizes, usually between 128-bits and 512-bits.

Easy to compute

- Hash functions are efficient and fast one-way functions.
- The requirement is that they be very quick to compute regardless of the message size.
- The efficiency may decrease if the message is too big but the function should still be fast enough for practical use.

- **Hash functions** are typically used to provide data integrity services.
- These can be used as one-way functions and to construct other cryptographic primitives, such as MACs and digital signatures.
- Some applications used hash functions as a means of generating **pseudo random numbers (PRNGs)**.
- Hash functions do not require a key.
- There are two practical and three security properties of hash functions that must be met depending on the level of requirements of integrity.

Pre-image resistance

- Consider an equation:

$$h(x) = y$$

- Here, h is the hash function, x is the input, and y is the hash.
- The first security property requires that y cannot be reverse computed to x . x is considered a *pre-image* of y , hence the name pre-image resistance. This is also called one-way property.
- It is a one-way property , it is easy to generate a code given a message , but virtually impossible to generate a message a given code.
- This property is important if the authentication techniques involves the use of a secret value. However , if the hash function is not one way , an attackers can easily discover the secrete value.

Second pre-image resistance

- This property requires that given x and $h(x)$, it is almost impossible to find any other message m , where $m \neq x$ and $\text{hash of } m = \text{hash of } x$.

$$h(m) = h(x).$$

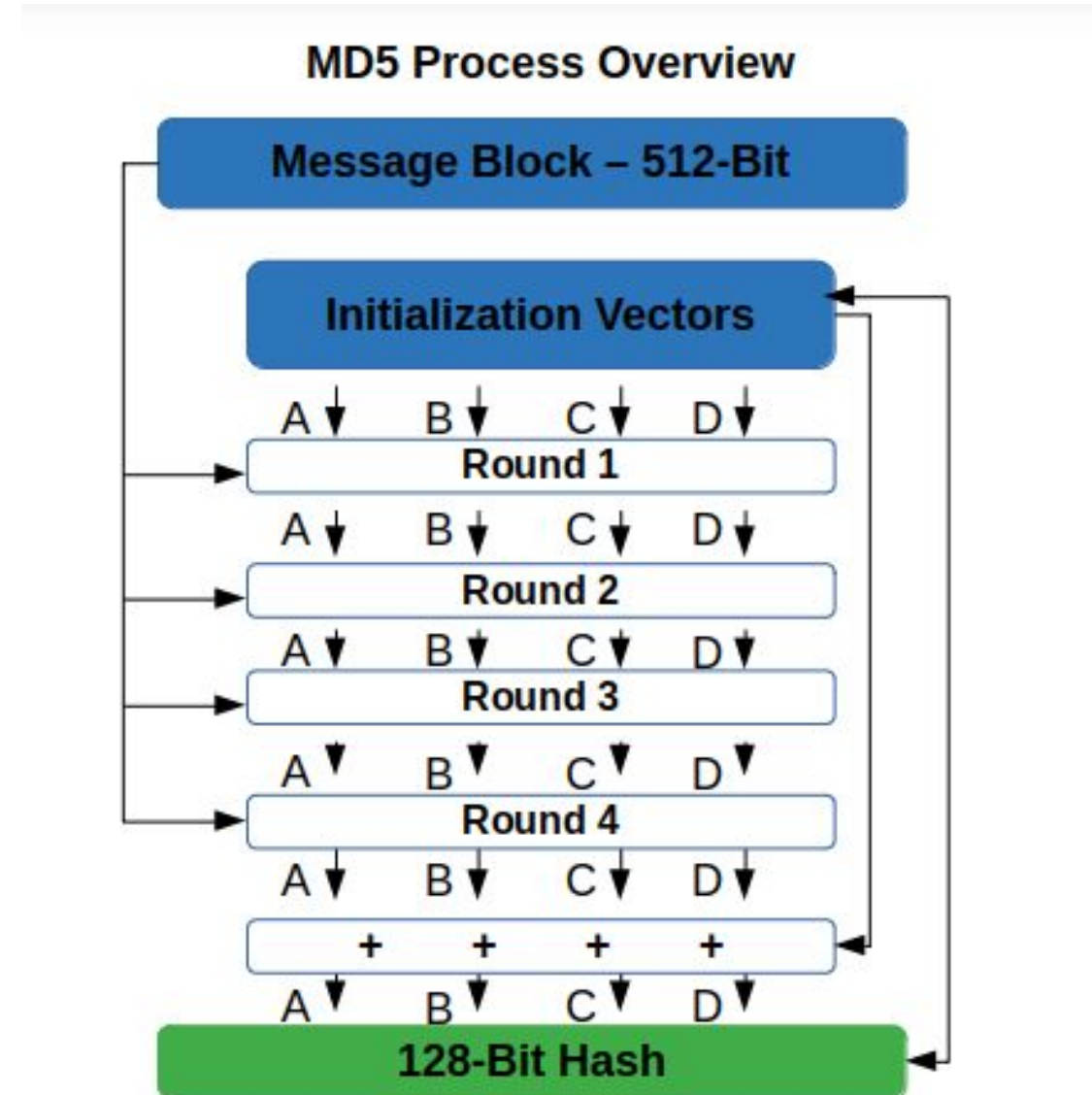
- This property is also known as weak collision resistance.
- It guarantees that it is impossible to find an alternative message with the same hash value as a given message.

Collision resistance

- This property requires that two different input messages should not hash to the same output.
- In other words, $h(x) \neq h(z)$. This property is also known as strong collision resistance.
- Hash functions, due to their very nature, will always have some collisions, and that is where two different messages hash to the same output, but they should be computationally infeasible to find. A concept known as avalanche effect is desirable in all hash functions.
- Avalanche effect specifies that a small change, even a single character change in the input text, will result in a totally different hash output.

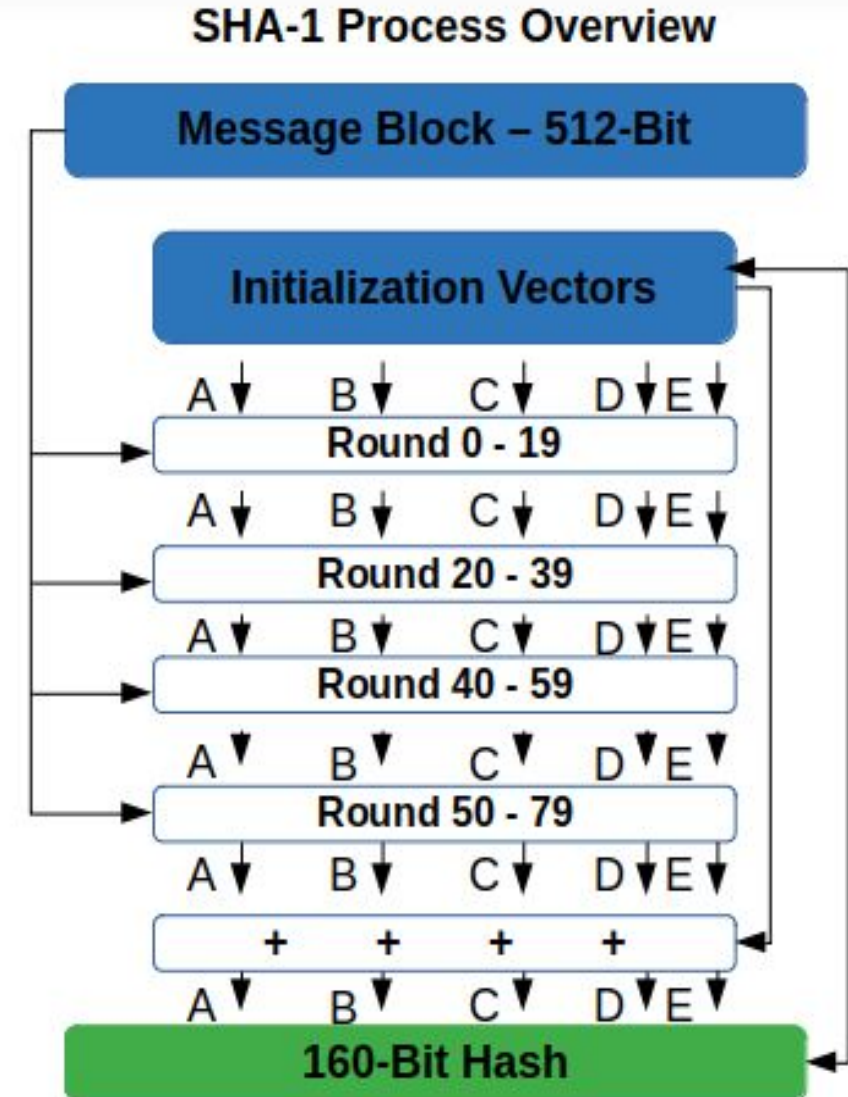
Message Digest (MD)

- Message Digest functions were very popular in early 1990s.
- MD4 and MD5 are members of this category.
- Both MD functions are found to be insecure and not recommended for use any more.
- MD5 is a 128-bit hash function that was commonly used for file integrity checks.



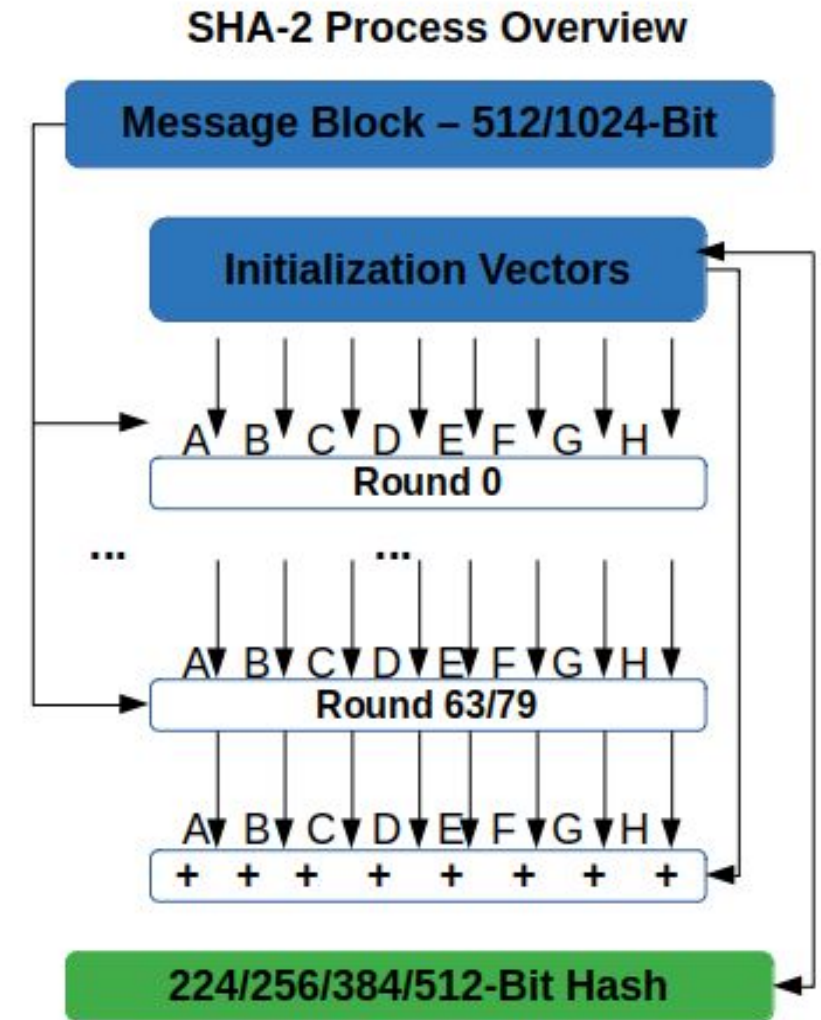
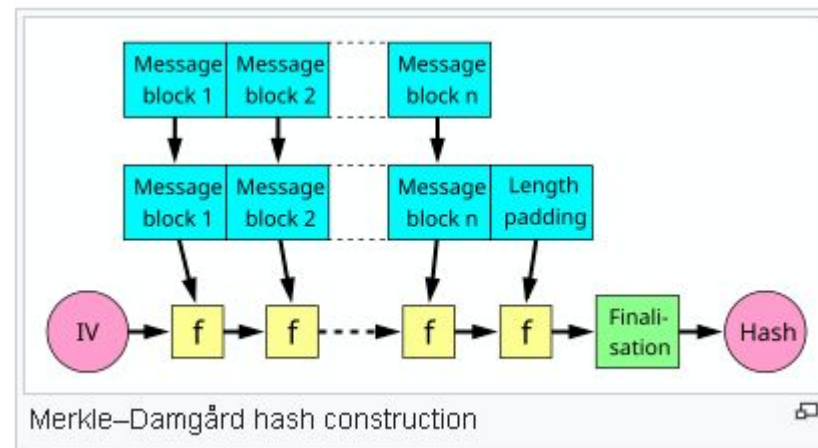
Secure Hash Algorithms (SHAs)

- **SHA-0:** This is a 160-bit function introduced by NIST in 1993.
- **SHA-1:** SHA-1 was introduced later by NIST as a replacement of SHA-0. This is also a 160-bit hash function.
- SHA-1 is used commonly in SSL and TLS implementations.
- It should be noted that SHA-1 is now considered insecure and is being deprecated by certificate authorities.
- Key length too short to resist to attacks.
- **Attack:** The "SHAttered" attack (2017) demonstrated a **collision** in SHA-1.



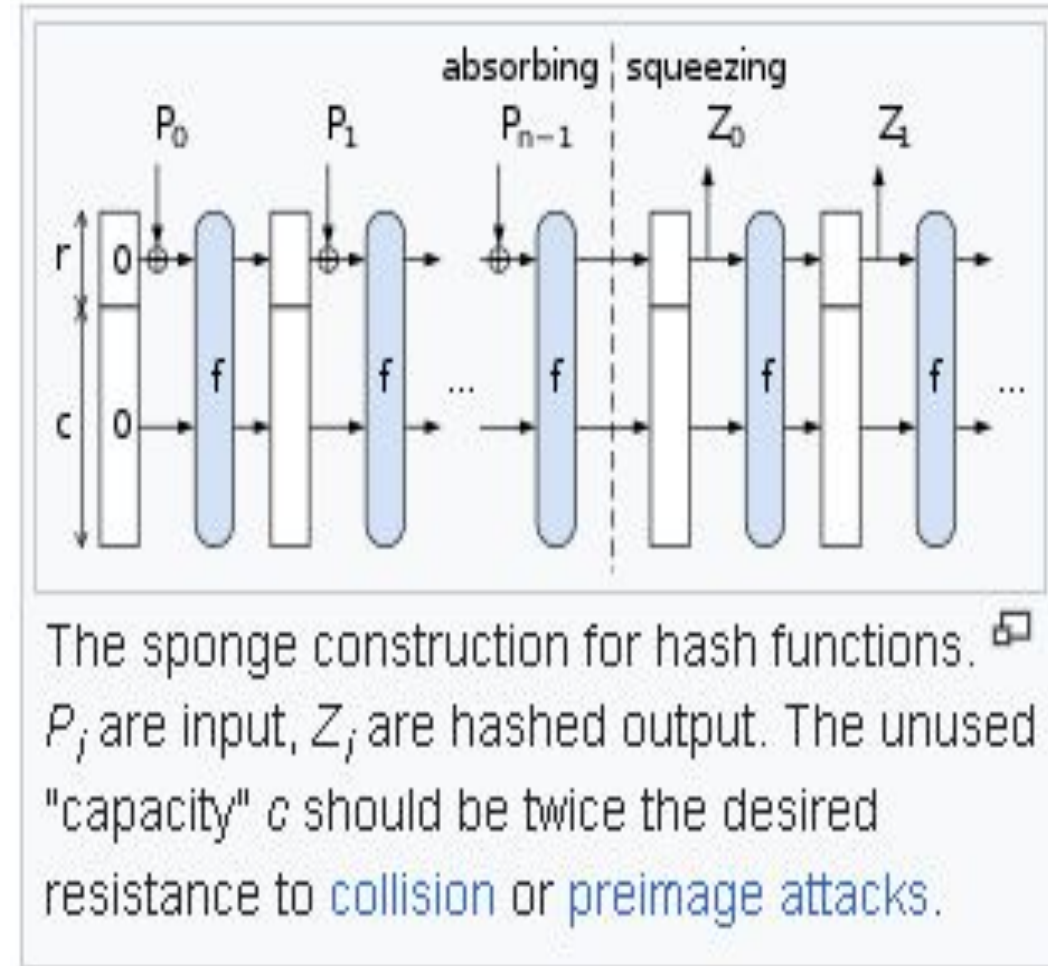
SHA-2 (Secure Hash Algorithm 2) is a set of cryptographic hash functions designed by the United States National Security Agency (NSA) and first published in 2001. They are built using the Merkle–Damgård construction, from a one-way compression function from a specialized block cipher.

This category includes four functions defined by the number of bits of the hash: SHA-224, SHA-256, SHA-384 and SHA-512.



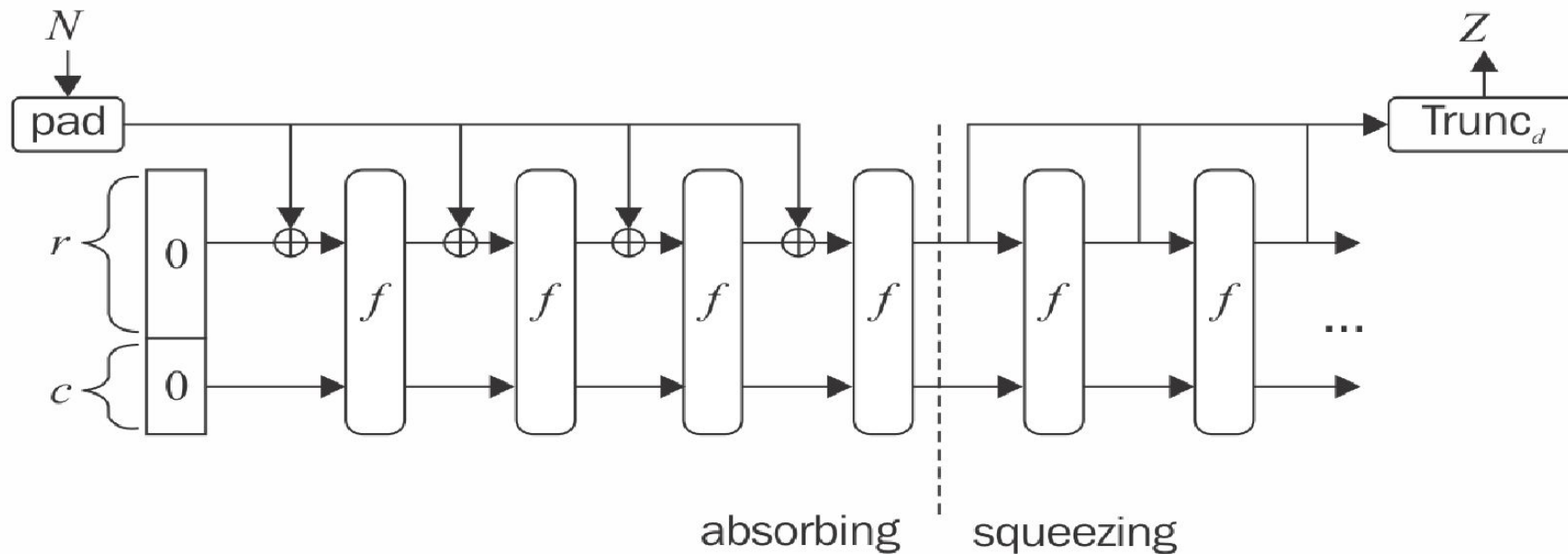
SHA - 3

- **SHA-3 (Secure Hash Algorithm 3)** is the latest member of the Secure Hash Algorithm family of standards, released by NIST on August 5, 2015.
- SHA-3 is a subset of the broader cryptographic primitive family Keccak.
- Keccak is based on a novel approach called sponge construction. Sponge construction is based on a wide random function or random permutation, and allows inputting ("absorbing" in sponge terminology) any amount of data, and outputting ("squeezing") any amount of data, while acting as a pseudorandom function with regard to all previous inputs. This leads to great flexibility.

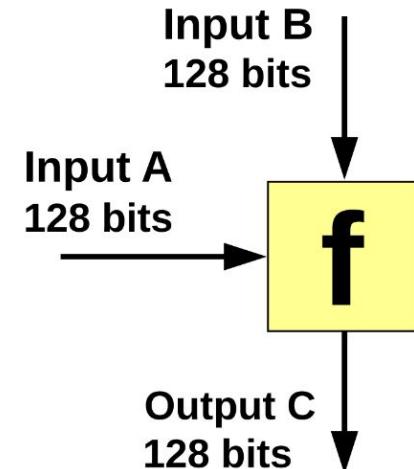


Design of SHA3 (Keccak)

- The following diagram shows the sponge and squeeze model that is the basis of SHA3 or Keccak.
- As an analogy to sponge, first, the data is absorbed into the sponge after applying padding, where it is then changed into a subset of permutation state using XOR and then the output is squeezed out of the sponge function that represents the transformed state.
- Rate is the input block size of a sponge function,
- whereas capacity determines the generic security level



- **RIPEMD:** RIPEMD is the acronym for *RACE Integrity Primitives Evaluation Message Digest*. It is based on the design ideas used to build MD4. There are multiple versions of RIPEMD, including 128-bit, 160-bit, 256-bit, and 320-bit.
- **Whirlpool:** This is based on a modified version of Rijndael cipher known as W. It uses the Miyaguchi-Preneel compression function, which is a type of one-way function used for the compression of two fixed length inputs into a single fixed length output. It is a single block length compression function



- Hash functions have many practical applications ranging from simple file integrity checks and password storage to be used in cryptographic protocols and algorithms.
- They are used in hash tables, distributed hash tables, bloom filters, virus finger printing, peer-to-peer P2P file sharing, and many other applications.
- In blockchain, hash functions play a very vital role. Especially, the proof of work function uses SHA-256 twice in order to verify the computational effort spent by miners.
- RIPEMD 160 is used to produce bitcoin addresses.

SHA-256

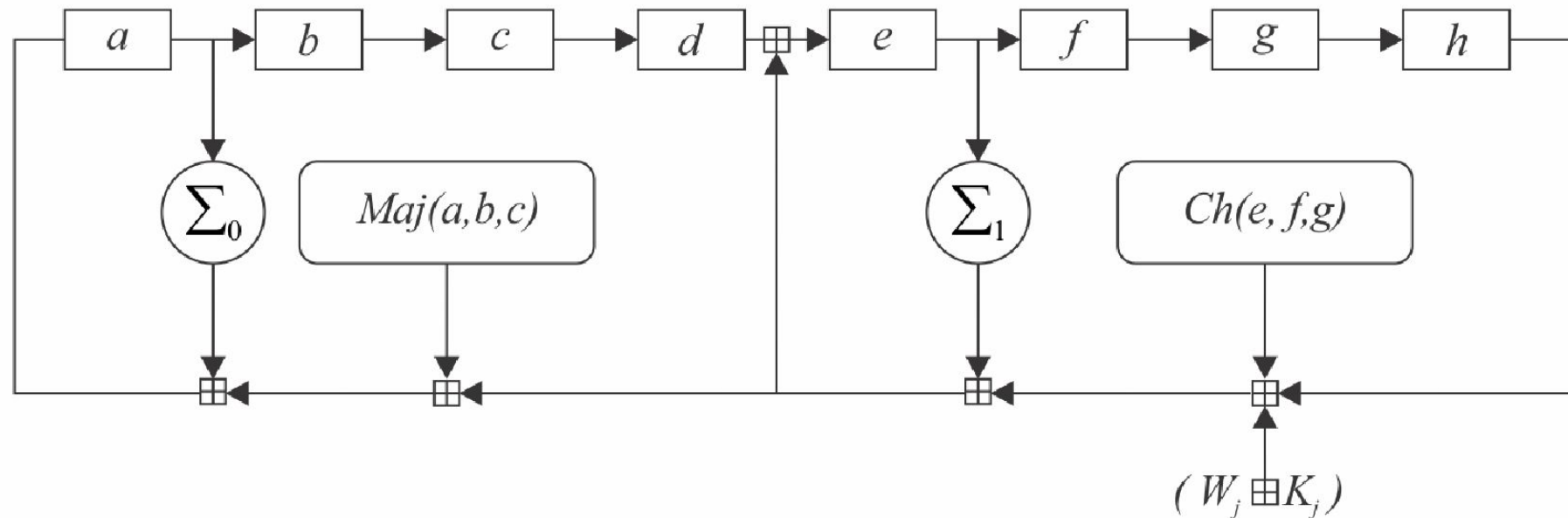
- SHA-256 has the input message size $< 2^{64}$ -bits. Block size is 512-bits and has a word size of 32-bits.
- Output is 256-bit digest.
- The compression function processes a 512-bit message block and a 256-bit intermediate hash value.
- There are two main components of this function:
 - compression function and
 - A message schedule.

- The algorithm works as follows:
- Pre-processing:
 1. Padding of the message, which is used to make the length of a block to 512-bits if it is smaller than the required block size of 512-bits.
 2. Parsing the message into message blocks that ensure that the message and its padding is divided into equal blocks of 512-bits.
 3. Setting up the initial hash value, which is the eight 32-bit words obtained by taking the first 32-bits of the fractional parts of the square roots of the first eight prime numbers.

These initial values are randomly chosen in order to initialize the process and gives a level of confidence that no backdoor exists in the algorithm.

- Hash computation:

1. Each message block is processed in a sequence and requires 64 rounds to compute the full hash output. Each round uses slightly different constants to ensure that no two rounds are the same.
2. First, the message schedule is prepared.
3. Then, eight working variables are initialized.
4. Then, the intermediate hash value is calculated.
5. Finally, the message is processed and the output hash is produced:



Understanding the SHA-256 Hash Computation

SHA-256 processes data in **64 rounds** using a series of logical operations on eight working variables (a , b , c , d , e , f , g , h). Each round updates these variables based on a **message schedule** (w_j) and **round constants** (K_j).

Key Components in the Diagram

1. σ_0 (Sigma 0) Function:

- Computes a rotated and XORed version of a to add diffusion.
- Formula:

$$\Sigma_0(a) = \text{ROTR}^2(a) \oplus \text{ROTR}^{13}(a) \oplus \text{ROTR}^{22}(a)$$

2. Maj (Majority) Function:

- Determines the majority bit among a , b , and c .
- Formula:

$$\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$$

3. σ_1 (Sigma 1) Function:

- Computes a rotated and XORed version of e to introduce randomness.
- Formula:

$$\Sigma_1(e) = \text{ROTR}^6(e) \oplus \text{ROTR}^{11}(e) \oplus \text{ROTR}^{25}(e)$$

4. Ch (Choose) Function:

- Selects bits from f and g based on the value of e .
- Formula:

$$\text{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g)$$

- This function helps SHA-256 introduce **non-linearity**.

5. Addition of w_j and K_j (Message Schedule and Round Constant):

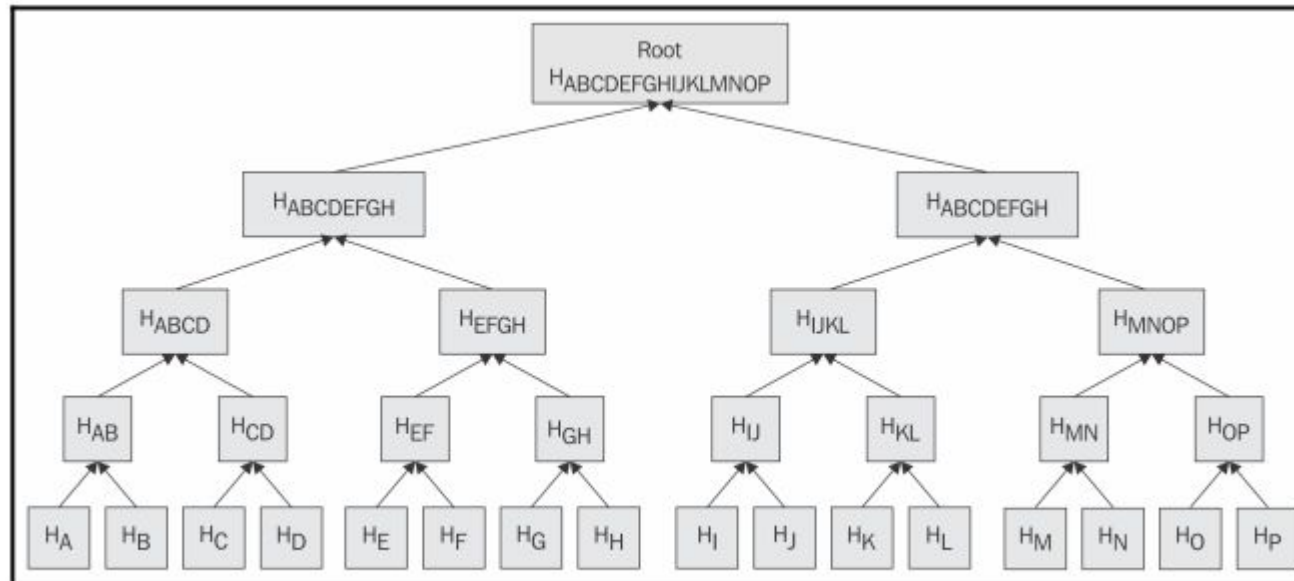
- w_j represents a **precomputed message schedule**.
- K_j is a predefined **round constant** used in SHA-256.

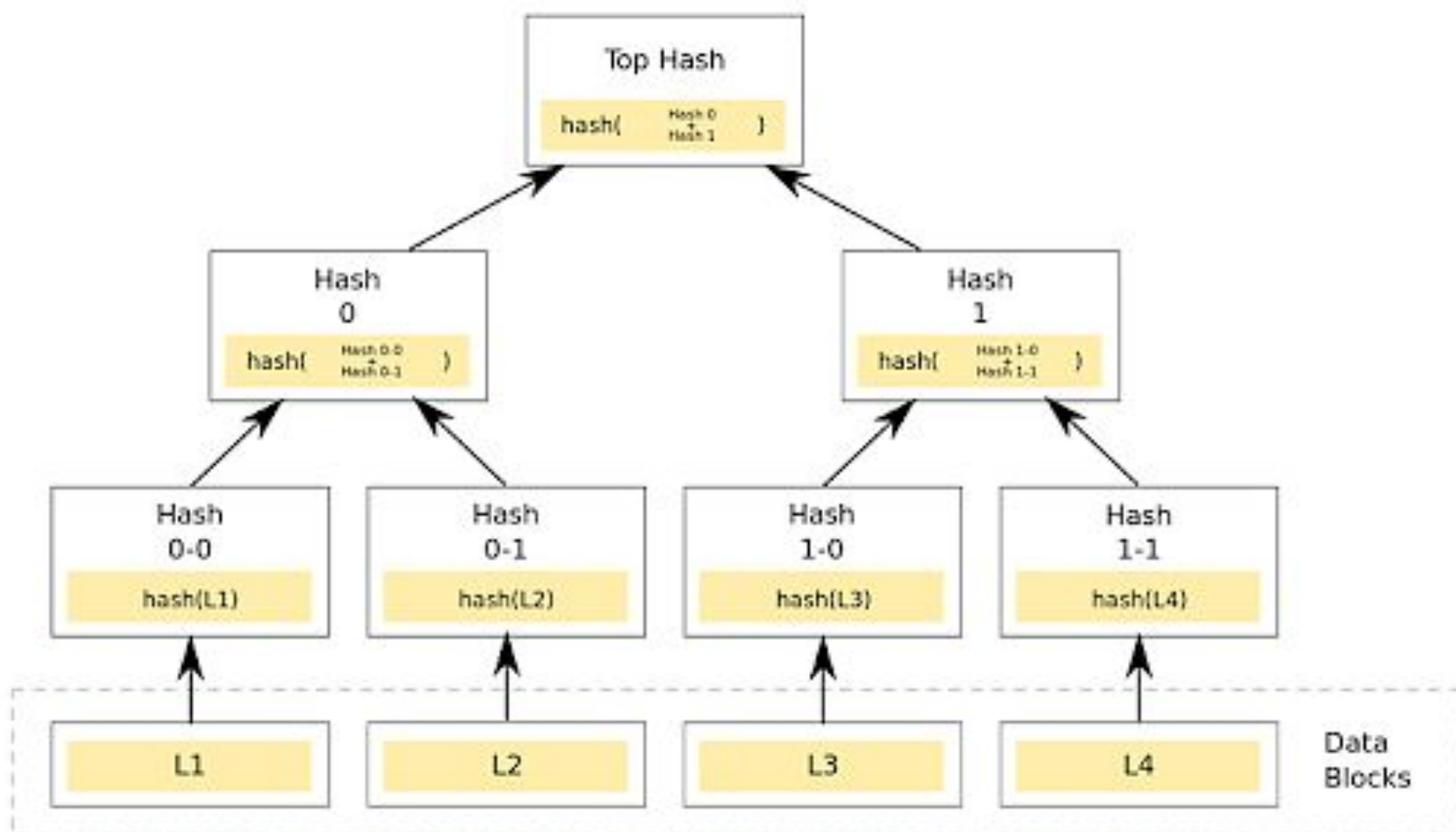
6. Update of Working Variables:

- The new value of d is updated by adding the result of the **right branch computation**.
- The new value of h is computed using contributions from $\Sigma_1(e)$, $Ch(e, f, g)$, w_j , and K_j .

Merkle trees

- Ralph(f) Merkle
- which makes it **easy to understand**.
- Merkle trees allow **secure and efficient verification of large data sets**.
- It is a **binary tree**
 - first, the **inputs** are placed at the leaves (node with no children)
 - then **values of pairs of child nodes are hashed together in order to produce a value for the parent node** (internal node) until a single hash value known as Merkle root is achieved:





- Bitcoin uses **Merkle Trees** to efficiently verify transactions within a block.

Scenario:

□ Imagine a **Bitcoin block** contains 4 transactions:

Tx1: Alice → Bob (1 BTC)

Tx2: Charlie → Dave (0.5 BTC)

Tx3: Eve → Frank (2 BTC)

Tx4: George → Henry (1.5 BTC)

□ Instead of storing all transactions separately, Bitcoin creates a **Merkle Tree**:

- ◆ Step 1: Hash Transactions

$H1 = \text{SHA-256}(Tx1)$

$H2 = \text{SHA-256}(Tx2)$

$H3 = \text{SHA-256}(Tx3)$

$H4 = \text{SHA-256}(Tx4)$

- ◆ Step 2: Pairwise Hashing

$H12 = \text{SHA-256}(H1 + H2)$

$H34 = \text{SHA-256}(H3 + H4)$

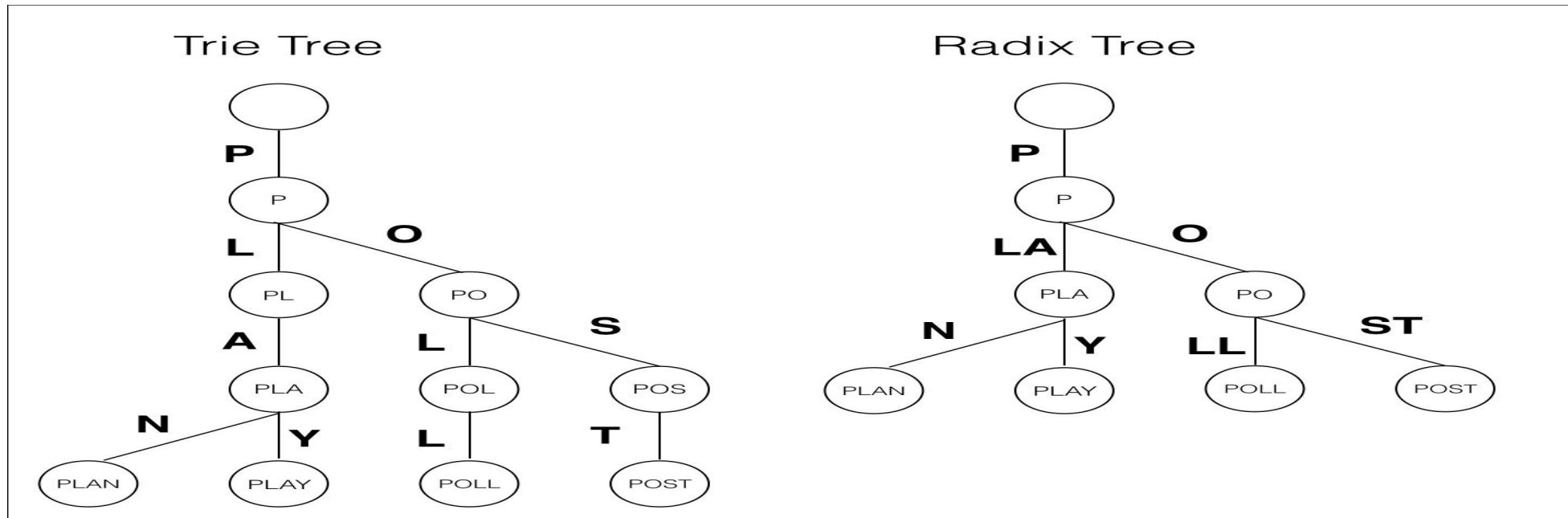
- ◆ Step 3: Compute Merkle Root

$\text{Merkle Root} = \text{SHA-256}(H12 + H34)$

□ The **Merkle Root** is stored in the **Bitcoin block header**.

Patricia trees

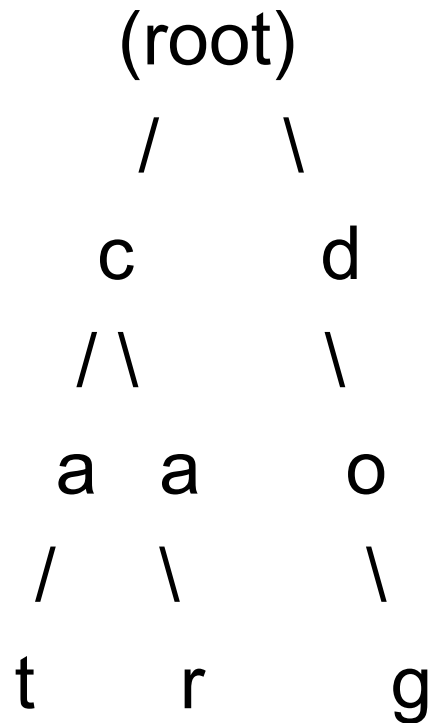
- To understand Patricia trees – know the concept of a trie.
- A **trie or a digital tree** is an **ordered tree data structure** used to store a dataset.
- **Practical Algorithm to Retrieve Information Coded in Alphanumeric (Patricia)**,
 - known as Radix tree,
 - is a **compact representation of a trie** in which a node that is the **only child of a parent** is merged with its parent.
- **Merkle-Patricia tree**, based on the definitions of Patricia and Merkle, is a tree that has a root node that contains the hash value of **the entire data structure**.



Example: Patricia Trie vs. Standard Trie

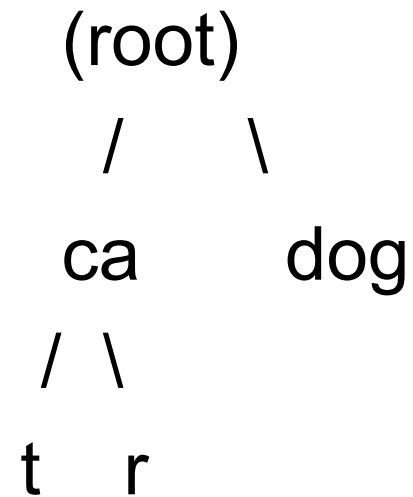
Example: Storing cat ,car, dog

Standard Trie



Each character is stored separately

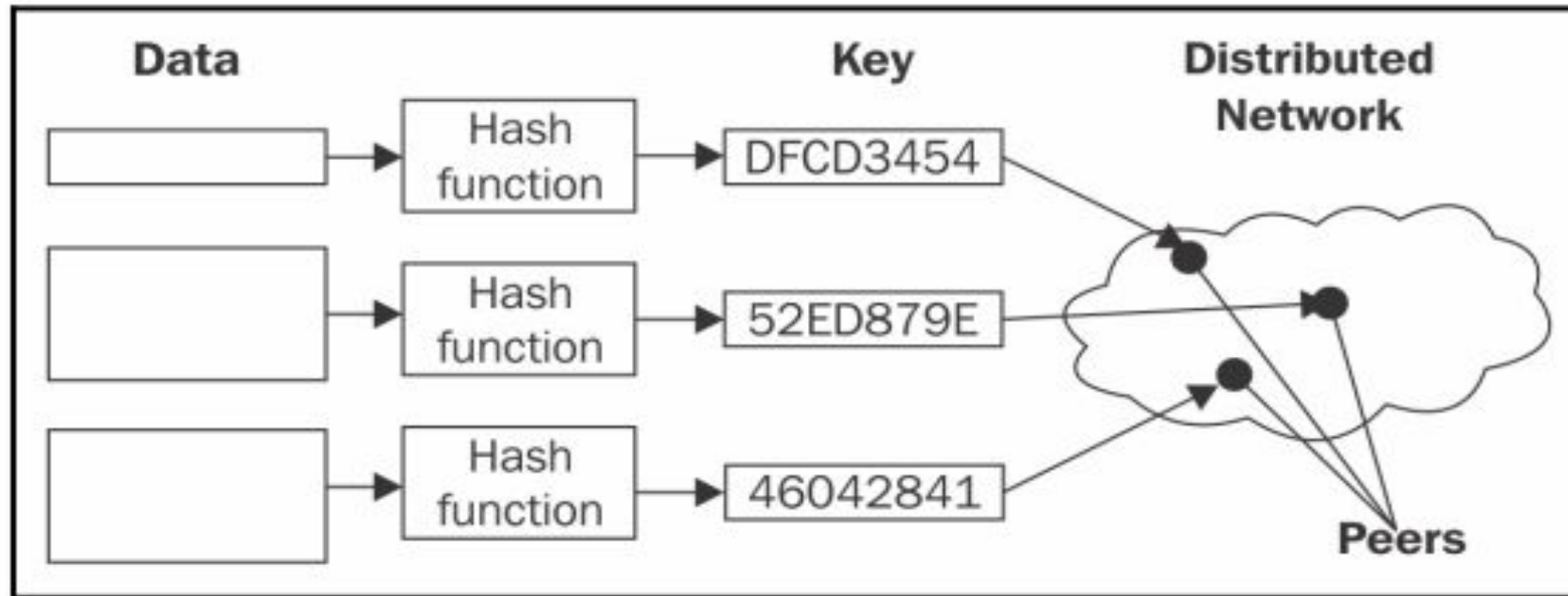
Example: Storing cat ,car, dog



Common prefixes ("ca") are stored as a **single node** instead of separate characters.

Distributed Hash Tables (DHTs)

- A hash table is a **data structure** that is used to **map keys to values**.
- Distributed hash table as a **data structure** where data is spread across various nodes and nodes are equivalent to buckets in **a peer-to-peer network**
- DHTs provides decentralization, fault tolerance, and scalability



The diagram represents a **Distributed Hash Table (DHT)**, a key concept in **peer-to-peer (P2P) networks** like **BitTorrent**, **IPFS**, and **blockchain-based storage systems**.

1. **Data → Hash Function → Key**

- Raw data is processed using a **cryptographic hash function** (e.g., SHA-256, MD5).
- The hash function produces a unique **key (hash value)** for the data.
- This key is used to store and retrieve the data efficiently.

2. **Key → Distributed Network**

- The generated key is used to locate data within a **distributed peer-to-peer network**.
- Instead of storing all data on a **single server**, it is **distributed across multiple peers (nodes)**.
- Each **peer stores a portion of the data**, and lookup requests are **routed to the correct node**.

Example: Storing a File in IPFS

1. A user uploads a file.
2. **IPFS hashes the file** and generates a **content-based address (CID)**.
3. **The file is distributed among multiple nodes** in the network.
4. **Retrieval:** Any user can request the file using its **CID**, and the network finds the nearest node storing it.

Digital Signature

- A digital signature is **an authentication mechanism** that enables the creator of a message to attach a code that **acts as a signature**.
- The signature is formed by **taking the hash** of the message and encrypting the message with the creator's private key.
- The signature guarantees the **source** and **integrity** of the message.
- The digital signature standard (DSS) is an NIST standard that **uses the secure hash algorithm (SHA)**.

Digital signatures are used to provide data origin **authentication and nonrepudiation.**

Properties:

- It must verify the **author and the date and time** of the signature.
- It must **authenticate the contents at the time** of the signature.
- It must be **verifiable by third parties**, to resolve disputes.

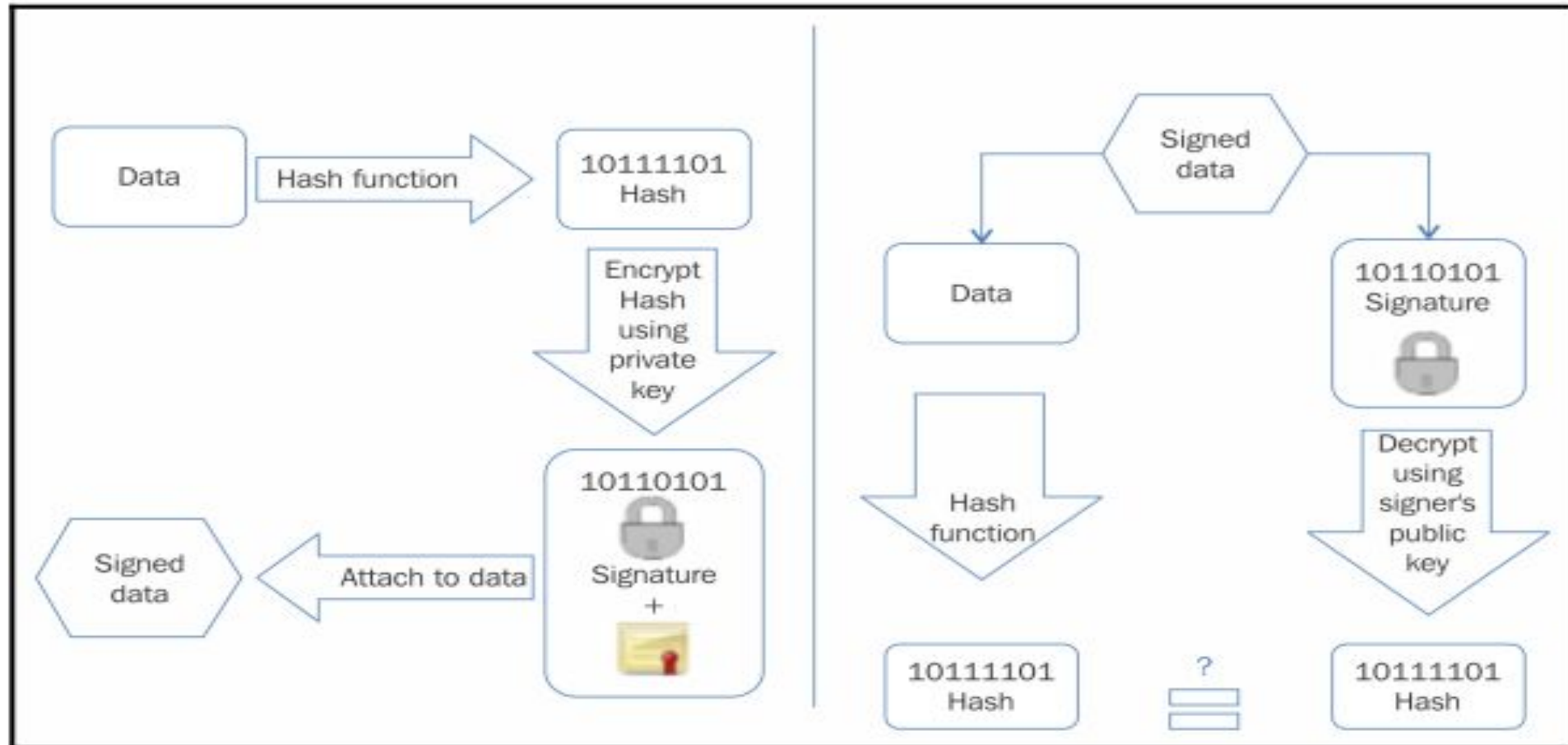
Thus, the digital signature function **includes the authentication function**

Important properties: authenticity, unforgeability, and non-reusability.

Authenticity: digital signatures are verifiable by a **receiving party**.

Unforgeability: ensures that only the **sender of the message** is able to use the signing functionality using the **private key**.

Non reusability: Digital signature **cannot** be separated from a message and used for another message again.



Two approaches to use digital signatures with encryption

Sign then encrypt

The sender digitally signs the data using the private key, appends the signature to the data, and then encrypts the data and the digital signature using the receiver's public key.

Encrypt then sign:

The sender encrypts the data using the receiver's public key and then digitally signs the encrypted data

Scenario

- A **defence command center** needs to ensure that messages sent to soldiers in the field are **authentic** and have not been altered by adversaries.

Solution : **Digital signatures** for verification.

Create a two folders Defence and soldier

- **Step 1: Generate RSA Key Pair**

The command center first generates an RSA key pair (**private & public key**).

Generate a 4096-bit RSA private key

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt  
rsa_keygen_bits:4096
```

Extract the public key

```
openssl rsa -in private_key.pem -pubout -out public_key.pem
```

- **Step 2: Create a Message to be Sent**

Create a text file **message.txt** containing the command.

```
echo "Attack at dawn. Coordinates: 45.1234, 76.5678" > message.txt
```

Step 3: Generate the Digital Signature

The command center signs the message using the **private key**.

**openssl dgst -sha256 -sign private_key.pem -out signature.bin
message.txt**

- It creates a digital signature (signature.bin) provides the authenticity

Step 4: Transmit the Message and Signature

- Now, send **message.txt**, **public key** and **signature.bin** to the soldier.

Step 5: Verify the Message at the Receiver's End

The soldier receives **message.txt** and **signature.bin**. To verify the message's authenticity, they use the **public key**.

```
openssl dgst -sha256 -verify public_key.pem -signature  
signature.bin message.txt
```

Expected Output:

- If the message is **authentic** (not altered), OpenSSL will output

Verified OK

- If the message has been **tampered**, OpenSSL will output

Verification Failure